UNIVERSIDAD TECNOLÓGICA DE BOLÍVAR

Ingeniería de Sistemas y Computación — Inteligencia Artificial

Diseño e implementación de un agente inteligente

Documento de diseño ajustado, desarrollo del agente y pruebas contra las medidas de desempeño

FlightPredict-Agent — Agente de predicción de retrasos de vuelo

Presentado por:

Hasmet Gonzalez Perez — T00070395
Dylan Andrés Ecker López — T00078024

Docente: Edwin Alexander Puertas Del Castillo

Cartagena de Indias, 17 de octubre de 2026

Contenido

0. Tabla de cambios respecto a la entrega anterior · Fase 1. Selección justificada del agente (matriz de decisión) 1–8. Diseño del agente (corregido) · 9. Desarrollo del agente (arquitectura, tecnologías, entradas, salidas, diagramas de secuencia, diferencias, pruebas, limitaciones, plataforma desplegada, enlaces) · 10. Conclusiones · Referencias · Anexo A

0. Tabla de cambios respecto a la entrega anterior (Tarea 2)

Las modificaciones se hicieron al implementar el agente y al contrastar el diseño con las exigencias de la Tarea 3. La fila final recoge las observaciones escritas del docente a la Tarea 2.

Sección	Qué decía	Qué dice ahora	Por qué cambió
Portada	Un documento por integrante, con un solo autor cada uno.	Un único documento de la pareja, que corresponde al agente elegido.	La entrega es una sola por pareja; la elección se justifica en la Fase 1.
Nueva sección (Fase 1)	No existía.	Matriz de decisión ponderada, análisis de sensibilidad y justificación (antes del diseño).	Exigido por la actividad: la elección debe poder reconstruirse.
2. Tipo de agente	Comparaba la probabilidad con «un umbral definido» sin explicitar la función de utilidad.	Se define U(alertar) = P·β·d − (1−P)·C_FA y se explica que el umbral es el punto de equilibrio P* = C_FA/(C_FA+β·d), que varía con la duración esperada.	Un agente basado en utilidad debe comparar alternativas con una función de utilidad explícita y verificable en el código.
3. Tipos de ambiente (dinamismo)	El agente «debe recalcular» la probabilidad a medida que se acerca la hora del vuelo.	El agente emite una predicción por vuelo a T − 180 min; el recálculo periódico queda como trabajo futuro.	Se implementó solo la predicción a las 3 h; el recálculo no se pudo probar en el tiempo disponible.
5. Entradas y sensores	Historial BTS con 32 variables; clima desde una API externa.	Se agregan identificador de aeronave, número de tramo, llegada programada del tramo previo y reloj del sistema. Las fuentes se simulan (ver 9.3); se usan 15 características.	Sin el identificador de aeronave y la hora actual no se pueden calcular el retraso en cadena ni la antelación.
6. Salidas y actuadores	Tres actuadores (alerta, sugerencia, registro del resultado).	Cuarto actuador: «marcar para revisión manual» ante entradas inválidas o ventana menor a 3 h.	La sección 8 ya hablaba de marcar entradas defectuosas, pero no existía el actuador.
7. Medidas de desempeño	Una sola medida compuesta (exactitud ≥ 70 % con 3 h de antelación), con evaluación continua y revisión trimestral.	Cuatro medidas con fórmula y umbral: M1 exactitud, M2 antelación, M3 recall de vuelos retrasados y M4 manejo de fallas; 13 escenarios × 5 repeticiones.	Con ≈ 20 % de vuelos retrasados, predecir «nunca hay retraso» ya da ≈ 80 % de exactitud; hacía falta una medida de sensibilidad. La revisión trimestral no es verificable en la entrega.
8. Memoria y razonamiento	«Consultar el historial de vuelos similares» sin concretar cómo.	Se concreta como tasa histórica de retraso por aeropuerto de origen y franja horaria, incluida en la alerta como contexto; la memoria de corto plazo guarda el último retraso conocido por aeronave.	Se necesita una definición implementable y comprobable de cada componente.
9 → 10. Conclusiones	Conclusiones de viabilidad conceptual.	Pasa a ser la sección 10 y se actualiza con los resultados reales (incluida la medida M3 incumplida).	Se inserta la sección 9 «Desarrollo del agente» antes de las conclusiones.
Nueva sección 9	No existía.	Arquitectura, tecnologías, entradas, salidas, diagramas de secuencia UML, diferencias, pruebas, limitaciones y enlaces.	Exigido por la Tarea 3.
Nueva sección 9.9	No existía.	Plataforma desplegada: interfaz web en Streamlit Community Cloud que ejecuta el mismo código del agente, con casos de prueba sugeridos.	Exigida por la Tarea 3 (plataforma de despliegue accesible por enlace público).
Referencias	Russell y Norvig, IBM, BTS, PLOS ONE.	Se agregan scikit-learn (Pedregosa et al., 2011) y CUAD (Hendrycks et al., 2021, usado en la matriz).	Tecnología utilizada y fuente de la comparación.
Observaciones del docente	—	No se nos proporcionaron observaciones	No se nos proporcionaron observaciones

Fase 1. Selección justificada del agente

Se comparan los dos agentes diseñados por la pareja: **Agente A — ClauseCheck-Agent** (revisión automatizada de contratos, basado en modelo con aprendizaje) y **Agente B — FlightPredict-Agent** (predicción de retrasos de vuelo, basado en utilidad con aprendizaje y modelo). Los criterios y los pesos se fijaron antes de puntuar.

Criterios y pesos

Criterio	Peso	Cómo se mide	Por qué ese peso
Viabilidad técnica	25 %	Componentes IBM que el equipo sabe implementar / 7	Es lo que más condiciona si el agente llega a existir en el tiempo de la entrega.
Disponibilidad de datos o entorno	20 %	Fuente identificada, simulador o dataset concreto	Sin entorno y datos no hay percepciones que probar; es importante, pero ambos agentes pueden simular el suyo.
Medibilidad del desempeño	20 %	Medidas con fórmula, umbral y verdad terreno objetiva	La actividad se califica con pruebas contra las medidas; por eso pesa igual que los datos.
Complejidad de IA	15 %	Tipo de agente según Russell y Norvig (reflejo simple → utilidad / aprendizaje)	Debe haber razonamiento real, pero la actividad no es un concurso de sofisticación.
Costo y recursos	10 %	Dinero, hardware, APIs pagas o cuotas	Importa al ser trabajo estudiantil sin presupuesto, pero no debe decidir por sí solo.
Valor del problema	10 %	Usuario y beneficio concretos	Aporta sentido al proyecto, pero ambos problemas tienen usuario claro.
Total	**100 %**		

Matriz de decisión ponderada

Criterio	Peso (w)	A: pts	A: evidencia (ClauseCheck)	B: pts	B: evidencia (FlightPredict)
Viabilidad técnica	25 %	3	El pipeline (extraer texto, segmentar cláusulas, clasificar con un modelo de lenguaje, registrar correcciones del abogado) es construible, pero exige PDF/OCR, un modelo de lenguaje y anotaciones; se estiman 5 de 7 componentes implementables sin infraestructura jurídica real (71 %).	4	Todo el flujo se resuelve con Python y scikit-learn en CPU; se estiman 6 de 7 componentes implementables (86 %); solo la integración con el sistema real de la aerolínea se simula.
Disponibilidad de datos o entorno	20 %	4	CUAD (conjunto público con contratos anotados por abogados) permite validar; el gestor documental se simula con archivos y la retroalimentación del abogado con las etiquetas del conjunto.	4	El entorno se simula por completo (clima, rotaciones, retrasos en cadena) y el historial BTS es público; unirlo con clima histórico exigiría otra fuente.
Medibilidad del desempeño	20 %	3	Una medida con fórmula y umbral (exactitud ≥ 80 % en 50 contratos), pero la verdad terreno depende de un abogado y «exactitud» es ambigua (por cláusula, categoría o fragmento).	4	La verdad terreno es objetiva y automática (el vuelo salió o no ≥ 15 min tarde); medida con fórmula y umbral. Debilidad: la exactitud sola es engañosa con ≈ 20 % de retrasos.
Complejidad de IA	15 %	4	Agente basado en modelo con componente de aprendizaje: comprensión semántica de texto y ajuste con correcciones.	5	Basado en utilidad con aprendizaje y modelo: estima probabilidad y duración, compara utilidades y mantiene estado de la aeronave (nivel superior en la escala de Russell y Norvig).
Costo y recursos	10 %	2	Requiere API de un modelo de lenguaje de pago (varias pasadas sobre 50 contratos largos) o GPU para ajuste fino, y OCR para escaneados.	5	Costo de 0 COP: datos simulados o públicos, scikit-learn en CPU, sin APIs de pago.
Valor del problema	10 %	4	Áreas jurídicas: reduce la lectura previa de cada contrato; no sustituye el criterio del abogado.	4	Equipos de operaciones de aerolíneas: anticipar retrasos en cadena y reasignar tripulaciones y puertas.
Puntaje ponderado S	**100 %**	**3,35**	S(A) = 0,25·3 + 0,20·4 + 0,20·3 + 0,15·4 + 0,10·2 + 0,10·4	**4,25**	S(B) = 0,25·4 + 0,20·4 + 0,20·4 + 0,15·5 + 0,10·5 + 0,10·4

Los puntajes de viabilidad son una estimación del equipo con base en lo descrito en cada diseño; quien aplique los mismos pesos y puntajes obtiene S(A) = 3,35 y S(B) = 4,25.

Análisis de sensibilidad

El criterio más influyente en la diferencia entre agentes es Costo y recursos ($w \times |pB - pA| = 0{,}10 \times 3 = 0{,}30$; le sigue Viabilidad técnica con 0,25). Se cambió su peso en ±10 puntos y los demás pesos se reescalaron proporcionalmente para que sumen 100 %; se repitió con Viabilidad técnica, el criterio de mayor peso.

Escenario	Pesos (Viab / Datos / Medib / Compl / Costo / Valor)	S(A) ClauseCheck	S(B) FlightPredict	Decisión
Base	25,0 / 20,0 / 20,0 / 15,0 / 10,0 / 10,0	3,35	4,25	B
Costo y recursos −10 pts (0 %)	27,8 / 22,2 / 22,2 / 16,7 / 0,0 / 11,1	3,50	4,17	B
Costo y recursos +10 pts (20 %)	22,2 / 17,8 / 17,8 / 13,3 / 20,0 / 8,9	3,20	4,33	B
Viabilidad técnica −10 pts (15 %)	15,0 / 22,7 / 22,7 / 17,0 / 11,3 / 11,3	3,40	4,28	B
Viabilidad técnica +10 pts (35 %)	35,0 / 17,3 / 17,3 / 13,0 / 8,7 / 8,7	3,30	4,22	B

La decisión no cambia en ninguno de los cuatro escenarios. Prueba extrema adicional: aun si ClauseCheck-Agent obtuviera 5 en Costo y recursos, S(A) sería 3,65 frente a 4,25 de B.

Justificación de la elección

El equipo elige **FlightPredict-Agent** con un puntaje ponderado de 4,25 sobre 5 frente a 3,35 de ClauseCheck-Agent, una diferencia de 0,90 puntos. La ventaja viene de cuatro criterios: costo (el agente de vuelos corre en CPU sin APIs de pago), viabilidad técnica (el flujo completo se resuelve con Python y scikit-learn), medibilidad (la verdad terreno es un hecho objetivo, no la opinión de un abogado) y complejidad (el agente de utilidad con aprendizaje está más arriba en la escala de Russell y Norvig). Empatan en disponibilidad de datos y en valor del problema. La sensibilidad muestra que la elección es estable: al mover ±10 puntos los pesos de Costo o de Viabilidad, B sigue ganando por al menos 0,67 puntos.

Qué se pierde al descartar ClauseCheck-Agent: la oportunidad de trabajar procesamiento de lenguaje natural sobre texto jurídico, un dominio con contratos anotados por expertos (CUAD) y un ciclo de retroalimentación humana más natural, y un problema con mayor carga ética (apoyo a la decisión de un abogado). A cambio, el equipo gana un agente que puede probarse de forma completa, repetible y con verdad terreno objetiva, que es lo que exige esta actividad.

1. Introducción y planteamiento del problema

Los retrasos en la operación aérea representan uno de los problemas más costosos y disruptivos para las aerolíneas, dado que afectan la asignación de tripulaciones, la disponibilidad de puertas de embarque y la experiencia de los pasajeros. Cuando un vuelo se retrasa, el efecto suele propagarse en cadena hacia los siguientes vuelos programados con la misma aeronave, generando un impacto operativo que crece con el tiempo si no se anticipa oportunamente. Actualmente, muchas aerolíneas reaccionan a los retrasos una vez que estos ya se han materializado, lo que limita su capacidad de reorganizar recursos de manera proactiva.

En este contexto, se propone FlightPredict-Agent, un agente inteligente orientado a estimar, con al menos tres horas de anticipación, la probabilidad de retraso de cada vuelo programado de una aerolínea piloto. Un agente inteligente constituye una solución pertinente porque permite integrar de manera autónoma múltiples fuentes de información heterogénea (meteorología, historial operativo, estado de la aeronave) y traducirlas en una recomendación accionable para el equipo de operaciones, sin depender exclusivamente del juicio manual de un analista humano bajo presión de tiempo.

2. Tipo de agente inteligente

Según la clasificación de Russell y Norvig (Artificial Intelligence: A Modern Approach), FlightPredict-Agent se clasifica principalmente como un **agente basado en utilidad con capacidad de aprendizaje**. Es basado en utilidad porque no se limita a decidir si existe riesgo: calcula una probabilidad P de retraso y una duración esperada d, y compara la utilidad esperada de las dos acciones posibles. Función de utilidad: **U(alertar) = P·β·d – (1 – P)·C_FA**, con U(no alertar) = 0, donde β = 0,05 es el valor de anticipar cada minuto de retraso y C_FA = 1 el costo de una falsa alarma. El agente alerta cuando U(alertar) > 0, equivalente a superar el umbral P* = C_FA/(C_FA + β·d); como P* depende de d, un retraso esperado más largo justifica alertar con menos certeza (por ejemplo, P* = 0,50 si d = 20 min y P* = 0,25 si d = 60 min).

Incorpora un componente de aprendizaje, dado que el modelo se entrena con el historial de vuelos y se retroalimenta con el resultado real de cada vuelo, y un componente basado en modelo, pues conserva una representación interna del estado de cada aeronave (el último retraso conocido) para inferir cómo ese estado, no observable directamente, afecta el vuelo actual.

3. Tipos de ambiente

Propiedad	Clasificación	Justificación
Observabilidad	Parcialmente observable	El agente solo percibe un subconjunto de variables (ruta, aerolínea, hora, pronóstico y último estado conocido de la aeronave); no ve la disponibilidad real de personal de tierra ni incidentes no reportados. En la implementación, el retraso del tramo previo solo se conoce si ese vuelo ya salió a T − 180 min.
Determinismo	Estocástico	La ocurrencia y la duración de un retraso dependen de factores probabilísticos (clima, congestión, tripulación); por eso el agente estima una probabilidad y no un resultado garantizado.
Episodicidad	Secuencial	La predicción de un vuelo depende del estado del vuelo anterior de la misma aeronave dentro de la jornada, de modo que las predicciones no son independientes.
Dinamismo	Dinámico	Clima, aeropuertos y aeronaves cambian mientras el agente delibera. Ajuste: la implementación emite una predicción por vuelo a T − 180 min; el recálculo periódico queda como trabajo futuro.
Discretización	Mixto, con predominio continuo	Hora, duración del retraso y magnitudes meteorológicas son continuas; aeropuerto, aerolínea y tramo son discretos.
Número de agentes	Agente único	Una sola entidad autónoma genera la predicción y la alerta; el sistema operativo y el equipo de operaciones forman parte del entorno.

4. Descripción del entorno

El agente opera dentro del contexto de una aerolínea piloto que sirve como caso de validación inicial. Los actores son el equipo de operaciones (reasigna tripulaciones y puertas), el sistema operativo de la aerolínea (con el cual el agente se integra para emitir alertas) y, de forma indirecta, los pasajeros. Entre las restricciones del dominio están la calidad de los pronósticos meteorológicos, la dependencia de fuentes históricas como el Bureau of Transportation Statistics (BTS) y la necesidad de predecir con al menos tres horas de antelación. En condiciones normales, el agente procesa vuelos con información estable; en condiciones excepcionales (tormentas severas, huelgas, fallas en cadena, registros incompletos o tardíos), el agente debe generalizar o, si no puede predecir, marcar el vuelo para revisión manual. Para la entrega, este entorno se reproduce con un simulador estocástico (sección 9).

5. Entradas y sensores

- **Aeropuerto de origen y destino:** datos categóricos del sistema de programación de vuelos.
- **Aerolínea:** dato categórico del sistema de programación.
- **Hora planeada del vuelo:** marca de tiempo del itinerario operativo.
- **Identificador de aeronave, número de tramo y llegada programada del tramo previo (agregado):** permiten calcular la holgura y el retraso en cadena.
- **Reloj del sistema (agregado):** permite calcular la antelación de cada predicción.
- **Condiciones meteorológicas previstas:** visibilidad, viento, precipitación y temperatura, captadas de un servicio meteorológico que actúa como sensor.
- **Estado del vuelo anterior de la misma aeronave:** en la implementación, el último retraso conocido en minutos, entregado por el sistema operativo cuando el vuelo sale.
- **Historial de vuelos y retrasos:** fuente de entrenamiento. Diseño: 32 variables del BTS. Implementación: historial simulado con 15 características.

6. Salidas y actuadores

- **Alerta al equipo de operaciones:** notificación estructurada al sistema operativo cuando U(alertar) > 0.
- **Sugerencia de reasignación proactiva:** ajuste de puerta, o de tripulación y puerta, hacia el sistema de gestión de recursos.
- **Registro del resultado real del vuelo:** escritura en el historial del agente (predicción + resultado) para el reentrenamiento.
- **Marcar para revisión manual (agregado):** señal al sistema operativo cuando el registro es inválido o la ventana es menor a tres horas; no se emite predicción.

7. Medidas de desempeño (SMART)

La medida original (exactitud ≥ 70 % con tres horas de antelación) se descompone en cuatro medidas medibles, cada una con fórmula y umbral fijados antes de ejecutar las pruebas (config/thresholds.json, fijado el 2026-10-16 14:40:04 UTC). Un vuelo se considera retrasado si sale 15 minutos o más después de lo programado.

Medida	Específica	Medible (fórmula)	Umbral	Alcanzable / Relevante	Temporal
M1 Exactitud	Clasificar vuelos retrasados vs. no retrasados	(VP + VN) / total	≥ 70 % (media global)	La literatura reporta niveles comparables; sin exactitud mínima las alertas pierden credibilidad.	50 corridas (10 escenarios × 5 rep.)
M2 Antelación	Emitir cada predicción con ≥ 3 h de antelación	predicciones con antelación ≥ 180 min / predicciones emitidas	100 %	Se controla con el disparador T − 180; sin ventana no hay reasignación posible.	65 corridas (13 escenarios × 5 rep.)
M3 Recall	Detectar los vuelos que realmente se	VP / (VP + FN)	≥ 60 % (media global)	Medida nueva: con ≈ 20 % de retrasos, la exactitud sola se logra	50 corridas (S01–S10)

Medida	Específica	Medible (fórmula)	Umbral	Alcanzable / Relevante	Temporal
	retrasan			sin detectar casi ninguno.	
M4 Manejo de fallas	No fallar ante entradas inválidas, ventana < 3 h o sensor sin dato	fallas manejadas / fallas inyectadas, sin excepciones	100 % por escenario	Un agente operativo no puede detenerse por un dato defectuoso.	15 corridas (S11–S13)

8. Componentes de la arquitectura de IA agéntica (marco IBM)

Fuente: IBM, «Components of AI agents» — https://www.ibm.com/mx-es/think/topics/components-of-ai-agents. Cómo implementa el agente cada uno de los siete componentes (el módulo de código correspondiente se indica en 9.1):

Percepción y manejo de entradas. Para cada vuelo, el agente recibe ruta, aerolínea, hora planeada, pronóstico y estado de la aeronave. Los datos se validan antes de ingresar al modelo: un pronóstico ausente o fuera de rango físico se imputa con la climatología y se marca; un registro inválido se marca para revisión manual, evitando que una entrada defectuosa distorsione la estimación.

Planificación y descomposición de tareas. A las tres horas de la salida, el agente descompone la tarea en cinco subtareas secuenciales: recopilar y validar entradas; consultar el historial de vuelos similares (tasa histórica de retraso por aeropuerto de origen y franja horaria); estimar probabilidad y duración con el modelo; comparar utilidades; y emitir alerta y sugerencia. El plan queda registrado en el log de cada decisión, lo que permite auditar la predicción.

Memoria. Memoria de largo plazo: historial de vuelos y resultados reales, climatología por aeropuerto y mes (para imputar) y tasas históricas; sirve de base para entrenar y reentrenar el modelo. Memoria de corto plazo: último retraso conocido de cada aeronave durante la jornada, necesaria para anticipar retrasos en cadena.

Razonamiento y toma de decisiones. Un modelo de aprendizaje automático estima la probabilidad de retraso y su duración esperada incorporando el retraso en cadena; la decisión de alertar se toma comparando las utilidades esperadas (sección 2), no con reglas fijas.

Llamada de herramientas y acción. Cuando U(alertar) > 0, el agente invoca la conexión con el sistema operativo de la aerolínea para enviar la alerta y la sugerencia; para obtener el pronóstico consulta un servicio meteorológico. En la implementación ambos servicios están simulados.

Comunicación. La alerta es un mensaje estructurado con identificador del vuelo, probabilidad, duración esperada, sugerencia de reasignación y tasa histórica de vuelos similares, dentro de la ventana de tres horas.

Aprendizaje y adaptación. Al salir cada vuelo, el resultado real se compara con la predicción emitida y se acumula como retroalimentación; el agente reentrena el modelo con el historial y las muestras nuevas (que pesan más), para adaptarse a cambios de patrón.

9. Desarrollo del agente

El agente implementado corresponde al diseño de las secciones 1 a 8: mismo tipo (basado en utilidad con modelo y aprendizaje), mismos sensores y actuadores, y un entorno simulado que respeta las propiedades de la sección 3 (parcialmente observable, estocástico, secuencial, dinámico, mixto, agente único). Las diferencias se documentan en 9.6. El ciclo percepción → decisión → acción está en src/flightpredict/agent.py (métodos perceive, decide, act y step).

9.1 Arquitectura

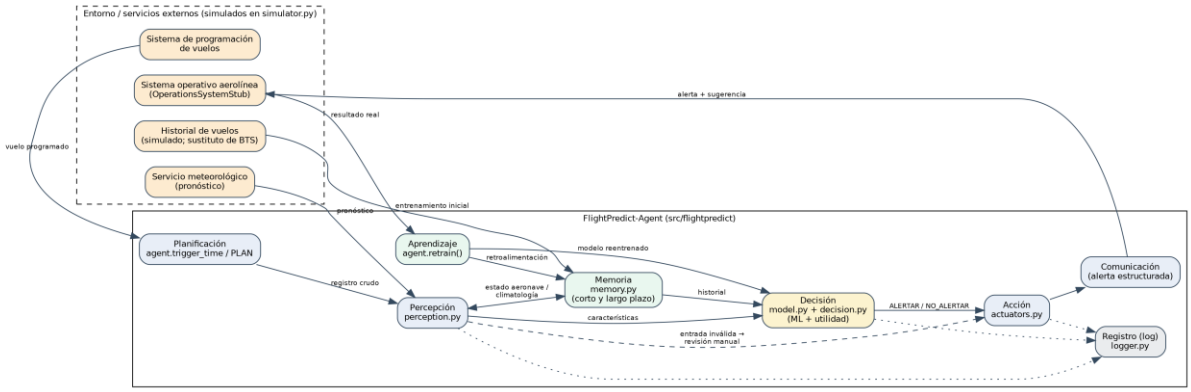


Figura 1. Diagrama de componentes: sensores y servicios externos (simulados), módulos del agente y flujo de datos. Línea punteada: registro (log); línea discontinua: ruta de excepción.

Componente IBM	Módulo de código (src/flightpredict)	Estado	Comentario
1. Percepción y manejo de entradas	perception.py: perceive(), extract_features()	Implementado	Valida campos, rangos y antelación; imputa clima; construye 15 características.
2. Planificación y descomposición	agent.py: trigger_time(), PLAN, step()	Parcial	Plan fijo de cinco pasos registrado en el log; no es un planificador dinámico.
3. Memoria	memory.py: AircraftMemory, LongTermMemory	Implementado	Corto plazo: estado por aeronave. Largo plazo: historial simulado, climatología, tasas.
4. Razonamiento y decisión	model.py: DelayModel; decision.py: decide()	Implementado	Modelo ML + función de utilidad. No se usa ningún LLM.
5. Llamada de herramientas y acción	actuators.py: Actuators, OperationsSystemStub	Simulado	No hay integración con un sistema real de aerolínea ni con una API meteorológica real.
6. Comunicación	actuators.py: send_alert()	Implementado	Mensaje estructurado (JSON); la interfaz web (streamlit_app.py, sección 9.9) permite operar el agente sin consola.
7. Aprendizaje y adaptación	agent.py: receive_outcome(), retrain()	Implementado	Reentrenamiento por lotes al acumular 150 muestras (probado en S10); no hay reentrenamiento trimestral automático.

9.2 Tecnologías

Qué razona el código y qué hace cada herramienta: la decisión (función de utilidad, validación, memoria, ciclo percepción → decisión → acción) la implementa el código propio; scikit-learn solo aporta el estimador de probabilidad y duración. No se usa ningún modelo de lenguaje.

Tecnología	Versión	Para qué se usó	Por qué frente a la alternativa
Python	3.12.3	Lenguaje del agente, el simulador y las pruebas	Ecosistema de ciencia de datos; alternativa Java, sin librerías ML equivalentes en el curso.
scikit-learn (HistGradientBoosting)	1.8.0	P(retraso) y duración esperada	Admite valores faltantes y variables categóricas de forma nativa y entrena en CPU en segundos; la regresión logística no captura interacciones (hora × congestión) y una red neuronal exige más datos y ajuste.
NumPy / pandas	2.4.4 / 3.0.2	Simulación estocástica y manejo	Estándar de facto; sin alternativas más simples para este

Tecnología	Versión	Para qué se usó	Por qué frente a la alternativa
		de tablas	volumen.
joblib	1.5.3	Guardar el modelo entrenado	Formato recomendado por scikit-learn para modelos.
matplotlib / Graphviz	3.10.8 / 2.43.0	Figuras de resultados y diagramas	Reproducibles desde script; las fuentes Mermaid (.mmd) y DOT (.dot) quedan en el repositorio.
python-docx	ver requirements-doc.txt	Generar este documento desde los resultados (scripts/construir_documento.py)	Las cifras del texto salen de los CSV, lo que evita transcribir mal.
Función de utilidad propia	—	Decisión de alertar	Hace explícito el compromiso entre falsas alarmas y retrasos anticipados; un umbral fijo no lo expresa.

9.3 Entradas (inputs)

Percepción	Tipo	Rango o formato	Origen	Ejemplo real (vuelo 1001-000192)
flight_id	texto	«semilla-número»	Sistema de programación	1001-000192
tail (aeronave)	texto	identificador no vacío	Sistema de programación	T100100045
leg (tramo)	entero	1 a 5	Sistema de programación	1
airline	categórico	9 códigos (p. ej. AA, DL, UA, XP)	Sistema de programación	NK
origin / dest	categórico	18 códigos IATA; origen ≠ destino	Sistema de programación	PHX / LAS
sched_dep	fecha-hora	ISO 8601; posterior a «now»	Sistema de programación	2026-07-09T10:25:52
prev_sched_arr	fecha-hora o nulo	ISO 8601	Sistema de programación	None
now (reloj)	fecha-hora	ISO 8601; antelación = sched_dep − now ≥ 180 min	Reloj del sistema	2026-07-09T07:25:52
forecast.vis	decimal	0 a 50 km	Servicio meteorológico (simulado)	9,7
forecast.wind	decimal	0 a 200 kt	Servicio meteorológico (simulado)	7,6
forecast.precip	decimal	0 a 500 mm	Servicio meteorológico (simulado)	0,0
forecast.temp	decimal	−60 a 60 °C	Servicio meteorológico (simulado)	25,0
Resultado real (retroalimentación)	decimal + fecha-hora	retraso en minutos; salida real	Sistema operativo de la aerolínea (simulado)	29,8 min
Historial de entrenamiento	tabla	31.984 vuelos simulados, 20,1 % retrasados	Simulador (sustituto del BTS)	models/history_features.csv.gz

9.4 Salidas y acciones

Acción (mensaje)	Cuándo se ejecuta	Contenido	Efecto en el entorno
ALERTA	U(alertar) > 0	flight_id, p_delay, d_hat_min, origin, tasa_historica_similares	El equipo de operaciones conoce un riesgo que aún no se materializa.
SUGERENCIA_ REASIGNACION	Junto con la alerta	REASIGNAR_PUERTA o REASIGNAR_TRIPULACION_Y_PUERTA (si d ≥ 45 min o retraso en cadena estimado ≥ 30 min)	El equipo puede reorganizar recursos antes del retraso.
REVISION_MANUAL	Entrada inválida o antelación < 180 min	flight_id, estado, problemas	El vuelo queda pendiente de revisión humana; no se emite predicción.
REGISTRO_RESULTADO	Al salir el vuelo	flight_id, p_delay, alerta, retraso real	Alimenta la memoria de la aeronave y el reentrenamiento.
Sin alerta (queda en el log)	U(alertar) ≤ 0	—	Ninguno; se registra la decisión con su utilidad.

9.5 Diagramas de secuencia UML

Participantes: actor externo (equipo de operaciones), entorno (simulador), interfaz (sistema operativo de la aerolínea), agente separado en planificación, percepción, memoria, decisión y acción, y el servicio externo (meteorológico). Los mensajes siguen el orden de las llamadas del código: trigger_time → perceive →

extract_features → predict → decide → act → receive_outcome → retrain. Fuente editable en Mermaid: docs/diagramas/secuencia.mmd.

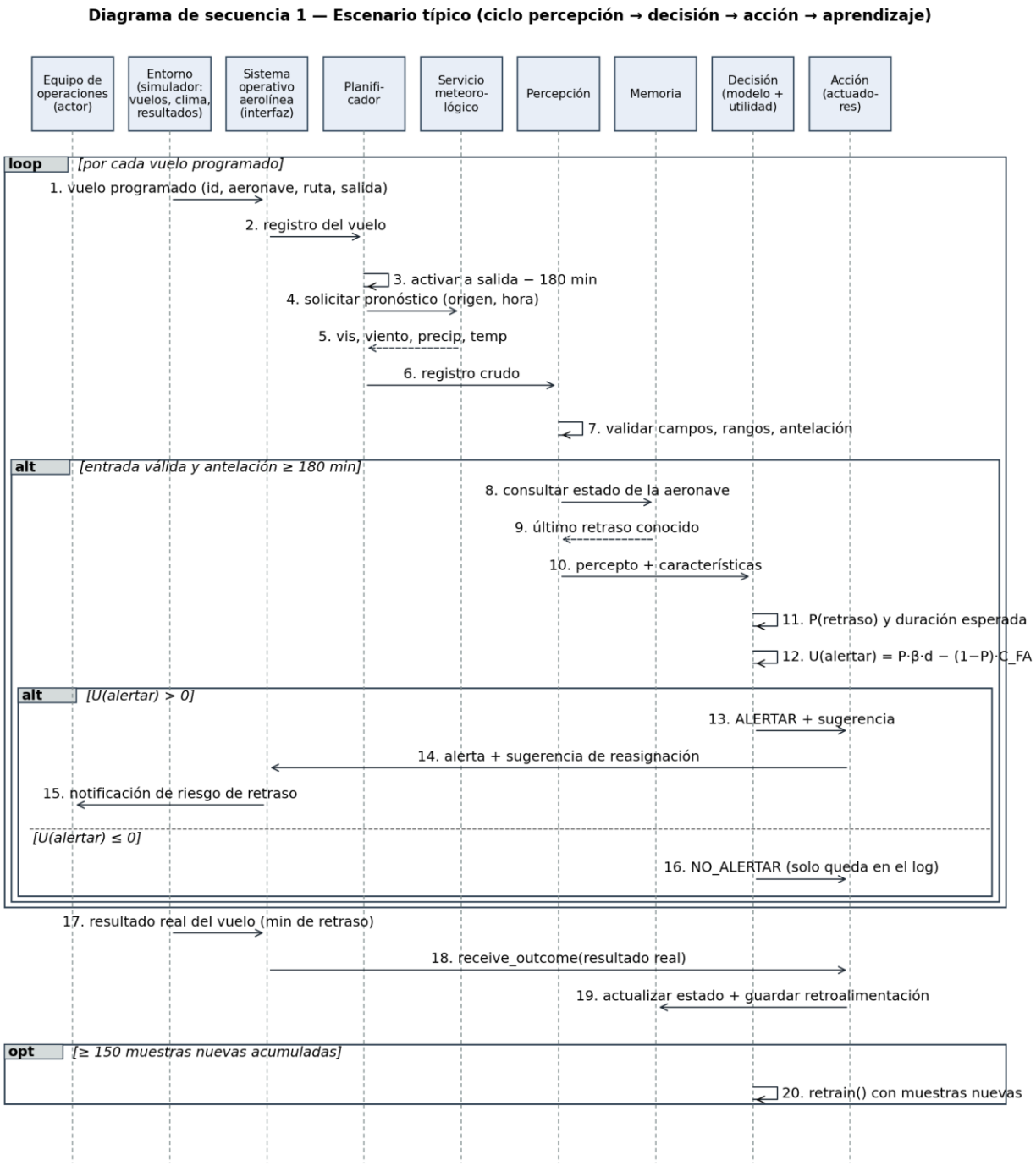


Figura 2. Secuencia del escenario típico, con fragmentos loop, alt (anidado) y opt.

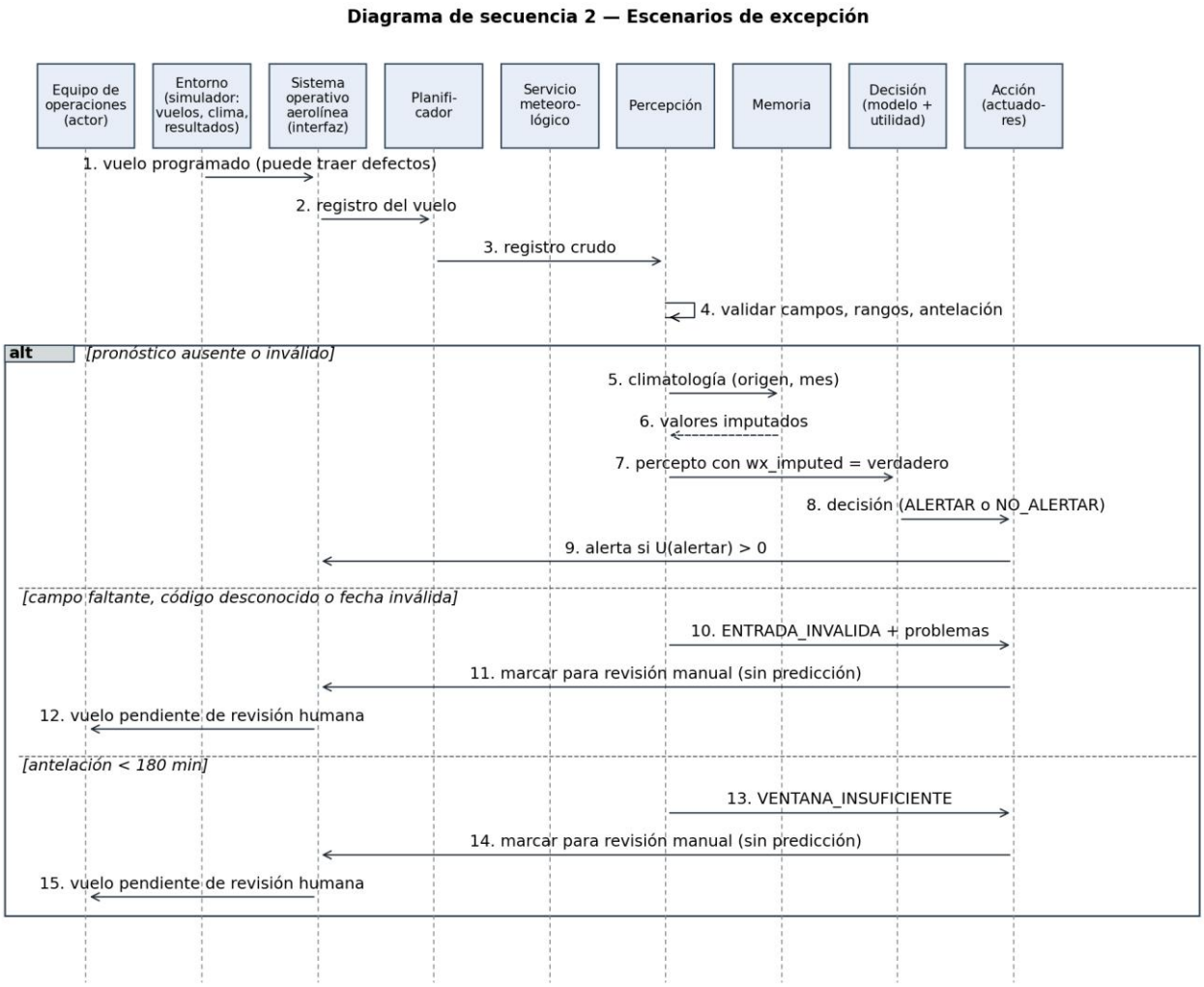


Figura 3. Secuencia de los escenarios de excepción: pronóstico ausente (imputación), entrada inválida y antelación insuficiente.

9.6 Diferencias entre diseño e implementación

Aspecto	Diseño	Implementación	Por qué
Entorno y datos	Historial BTS, API meteorológica y sistema operativo reales	Simulador estocástico con verdad terreno oculta; pronóstico con error	No hay acceso a datos ni sistemas reales; el simulador permite reproducibilidad y repeticiones.
Variables	32 variables del BTS	15 características (hora, día, mes, origen, destino, aerolínea, tramo, holgura, último retraso, retraso en cadena estimado, 4 variables de clima)	Son las que el simulador puede generar y las que el diseño declara como entradas.
Regla de decisión	Probabilidad contra un umbral definido	Utilidad explícita; el umbral es P* = C_FA/(C_FA + β·d)	Coherencia con un agente basado en utilidad.
Recálculo	Continuo, a medida que se acerca el vuelo	Una predicción a T − 180 min	Alcance de la entrega; queda como trabajo futuro.
Estado de la aeronave	Estado del vuelo anterior (a tiempo, retrasado, cancelado)	Último retraso conocido en minutos; no se modelan cancelaciones	Simplificación del simulador.
Aprendizaje	Reentrenamiento periódico (trimestral)	Reentrenamiento por lotes cada 150 resultados, con peso 5 para las muestras nuevas	Permite probar el ciclo de aprendizaje en una sola ejecución (escenario S10).
Actuadores	Tres	Cuatro (se agrega revisión manual)	Cubre entradas inválidas y ventana insuficiente.
Medidas de desempeño	Una medida compuesta	Cuatro medidas (M1–M4)	Ver tabla de cambios, sección 7.

9.7 Pruebas

Método. Cada medida se prueba en un entorno simulado con 13 escenarios (10 normales o límite y 3 de falla esperada); cada escenario se ejecuta 5 veces con semillas distintas (semilla = 1000·n.º de escenario + repetición), es decir, 65 corridas y 20.984 vuelos. Los umbrales se fijaron en config/thresholds.json antes de ejecutar (2026-10-16 14:40:04 UTC) y no se modificaron. El modelo se entrenó con 31.984 vuelos simulados con semilla distinta de la de las pruebas; el parámetro β = 0,05 se definió en el diseño y no se ajustó después de ver los resultados.
Reproducibilidad: los resultados de esta tabla se obtuvieron con las versiones de 9.2. Una reejecución independiente con Python 3.13 y scikit-learn 1.9.1 dio M1 = 74,0 %, M2 = 100 %, M4 = 100 % y M3 = 46,2 % (en lugar de 46,0 %): las conclusiones no cambian, pero las décimas varían con la versión de las librerías.

Tabla de resultados

Medida de desempeño	Fórmula	Umbral	Casos ejecutados	Resultado obtenido	¿Cumple?	Evidencia (log, archivo)
M1 Exactitud de clasificación (retrasado / no retrasado)	(VP + VN) / total; media de 50 corridas (S01–S10 × 5 rep.)	≥ 70 %	10 escenarios × 5 rep. = 50 corridas, 16509 vuelos	74,0% (DE 6,8 pts)	SÍ	results/detalle_por_corrida.csv, results/vuelos_detalle.csv.gz, results/logs/S01-S10
M2 Antelación de la predicción	predicciones con antelación ≥ 180 min / predicciones emitidas	≥ 100 %	13 escenarios × 5 rep. = 65 corridas, 19653 predicciones	100,0%	SÍ	columna lead_min de results/vuelos_detalle.csv.gz; fase «decision» de los logs
M3 Sensibilidad (recall) de vuelos retrasados	VP / (VP + FN); media de 50 corridas (S01–S10 × 5 rep.)	≥ 60 %	10 escenarios × 5 rep. = 50 corridas, 16509 vuelos	46,0% (DE 16,1 pts)	NO	results/detalle_por_corrida.csv (columna recall)
M4 Manejo de fallas (entrada inválida, ventana < 3 h, sensor sin dato)	casos de falla manejados correctamente / casos de falla inyectados (sin excepciones)	≥ 100 % en cada escenario de falla	3 escenarios de falla × 5 rep. = 15 corridas	S11 100,0%; S12 100,0%; S13 100,0%; excepciones: 0	SÍ	logs S11, S12 y S13 (fase «accion» = MARCAR_REVISION_MANUAL) y columnas M4_* del detalle

Escenarios de prueba (13): normales, límite y fallas esperadas

ID	Escenario	Tipo	Exactitud media ± DE	Recall media ± DE	Línea base (nunca alertar)
S01	Operación normal diurna	normal	77,7 % ± 2,2	38,4 % ± 8,5	81,2 %
S02	Hub congestionado en hora pico	normal	68,4 % ± 3,4	53,8 % ± 7,2	71,4 %
S03	Tormenta severa	límite	62,3 % ± 1,8	82,6 % ± 3,5	61,7 %
S04	Madrugada de bajo tráfico	límite	84,0 % ± 2,0	23,8 % ± 5,3	85,5 %
S05	Retrasos en cadena por incidentes	límite	73,1 % ± 4,0	47,1 % ± 5,9	65,4 %
S06	Aeropuertos poco frecuentes	límite	83,3 % ± 3,9	31,1 % ± 4,1	83,7 %
S07	Temporada alta de verano	normal	74,2 % ± 1,3	47,1 % ± 7,2	76,3 %
S08	Aerolínea poco vista	límite	75,3 % ± 3,0	40,6 % ± 6,9	78,2 %
S09	Invierno con clima adverso	normal	72,8 % ± 2,0	53,0 % ± 3,4	71,2 %
S10	Deriva operativa con retroalimentación	límite	69,3 % ± 2,6	42,1 % ± 3,0	65,2 %
S11	entradas inválidas	falla	79,2 % ± 2,9 (inf.)	41,9 % ± 13,0 (inf.)	79,6 %
S12	ventana menor a 3 h	falla	77,2 % ± 2,6 (inf.)	44,8 % ± 6,1 (inf.)	79,1 %
S13	sensor meteorológico sin dato	falla	79,0 % ± 1,1 (inf.)	36,0 % ± 7,7 (inf.)	80,9 %

(inf.) = informativa: en los escenarios de falla la medida que se evalúa es M4, no M1/M3.

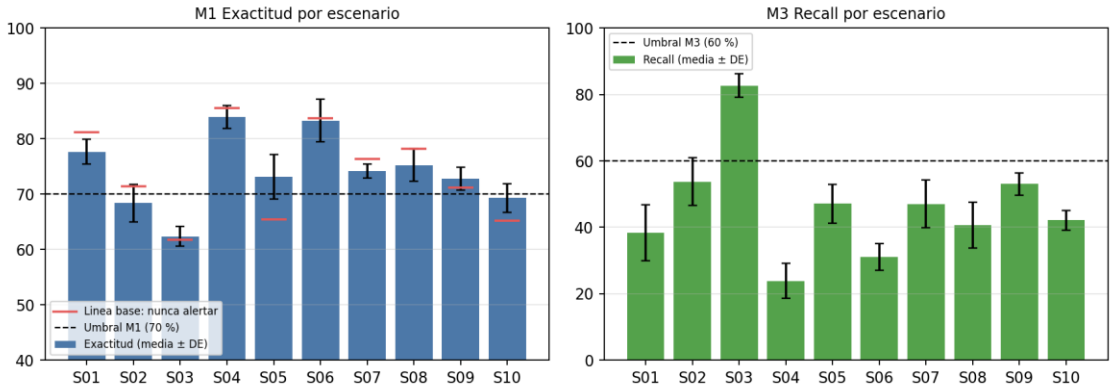


Figura 4. Exactitud y recall por escenario (media ± desviación estándar sobre 5 repeticiones). Los marcadores rojos son la exactitud de la línea base «nunca alertar».

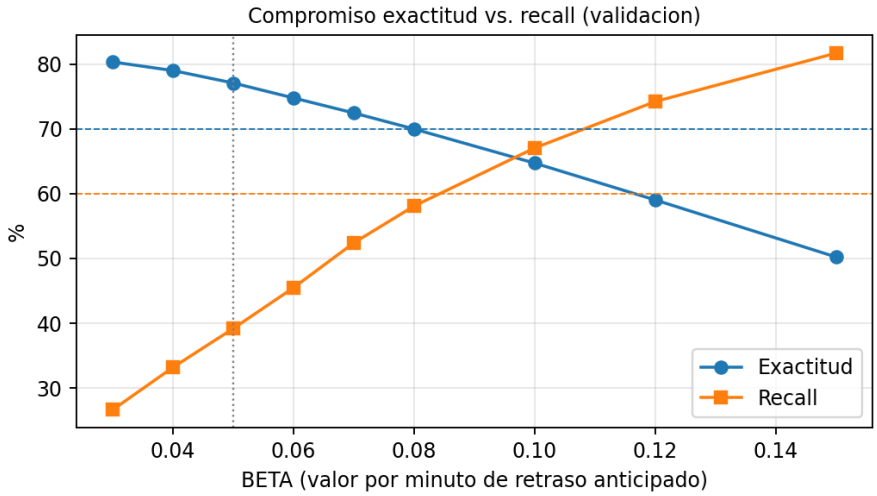


Figura 5. Compromiso exactitud–recall al variar β (datos de validación). La línea vertical marca el valor usado (β = 0,05).

Análisis de resultados

M1 (exactitud): cumple, con una advertencia. La media global es 74,0 % (DE 6,8 puntos; umbral 70 %). Tres escenarios quedan por debajo de 70 % por separado: S02 (68,4 %), S03 (62,3 %), S10 (69,3 %). La advertencia: en 6 de los 10 escenarios la exactitud del agente es menor que la de la línea base «nunca alertar» (p. ej. S01: 77,7 % frente a 81,2 %), porque con ≈ 20 % de retrasos esa línea base ya supera el umbral. M1 se cumple, pero por sí sola no demuestra que el agente aporta valor; por eso se añadió M3.

M2 (antelación): cumple (100 %). Ninguna de las predicciones emitidas tuvo menos de 180 min. Es cierto por construcción del disparador T − 180 y por la validación de ventana: en S12 los registros tardíos se derivaron a revisión manual en lugar de predecirse. Con datos reales, la antelación dependerá también de cuándo llegue el registro desde el sistema de programación.

M3 (recall): NO cumple, 46,0 % frente a un umbral de 60 %. La dispersión entre escenarios es grande (de 23,8 % en la madrugada a 82,6 % en tormenta). Causas probables: (1) con β = 0,05 y C_FA = 1 el agente alerta solo si P supera ≈ 0,25–0,40 y el modelo rara vez llega a esa probabilidad en retrasos «ordinarios»; (2) observabilidad parcial: el retraso del tramo previo solo se conoce si ese vuelo ya salió a T − 180 min y los incidentes ocultos son impredecibles; (3) el pronóstico tiene error. Evidencia en validación (datos distintos de los de prueba): con β = 0,08 el recall es 58,1 % y la exactitud 70,0 %; con β = 0,10 el recall es 67,0 % y la exactitud 64,7 %. Con este modelo, M1 ≥ 70 % y M3 ≥ 60 % no pueden cumplirse a la vez: hay un compromiso entre falsas alarmas y retrasos detectados (Figura 5). **Qué**

cambiaríamos: (a) dar más información al modelo (recalcular a T − 90 y T − 60 min con el estado del vuelo previo ya conocido y usar datos reales con más variables); (b) acordar con operaciones cuánto cuesta una falsa alarma frente a un retraso no anticipado y fijar β y C_FA en consecuencia; (c) reformular M1 y M3 como una sola medida (F1 o costo esperado) con un umbral nuevo justificado antes de volver a probar.

M4 (manejo de fallas): cumple (100 % en S11, S12 y S13, 0 excepciones). Los registros corruptos y los tardíos se marcaron para revisión manual sin marcar ningún registro válido por error, y los vuelos con pronóstico ausente recibieron una predicción con clima imputado y bandera. **Aprendizaje (S10):** el agente se reentrenó 3 o 4 veces por corrida; la diferencia entre el primer y el último cuarto del flujo es del orden de la desviación entre repeticiones, por lo que no se puede afirmar que el reentrenamiento mejore el desempeño, solo que el ciclo de retroalimentación funciona de extremo a extremo.

9.8 Limitaciones y trabajo futuro

- Los resultados son contra un simulador escrito por el equipo: reflejan la lógica del agente, no su desempeño en el tráfico aéreo real. Con datos del BTS las cuatro medidas deben repetirse.
- El simulador se calibró para una tasa de retrasos cercana al 20 % (la tasa histórica típica del BTS); si es más o menos predecible que la realidad, M1 y M3 cambiarían.
- Una sola predicción a T − 180 min; falta el recálculo a medida que se acerca el vuelo y modelar cancelaciones.
- Sin integración real con un sistema operativo de aerolínea ni con una API meteorológica.
- La interfaz web desplegada es una demostración sobre el simulador (estado en memoria, sin base de datos ni autenticación); no es un panel operativo conectado a una aerolínea real.
- Trabajo futuro: reevaluación periódica, función de utilidad calibrada con costos reales, calibración de probabilidades, análisis de equidad por aeropuerto y aerolínea, y conexión a datos reales (BTS y una API meteorológica).

9.9 Plataforma desplegada

El agente se publicó como aplicación web en **Streamlit Community Cloud** (el primer intento, Hugging Face Spaces con Gradio, se descartó por un error 403 de cuota de CPU). La interfaz (streamlit_app.py y core.py) **no reimplementa el agente**: importa el mismo paquete src/flightpredict, el mismo simulador y el mismo scripts/run_tests.py, de modo que lo que se ve en la web es lo que se ejecuta desde consola. Al iniciar carga models/delay_model.joblib; si la versión de scikit-learn no coincide, reentrena el modelo una sola vez (≈ 1 min) y lo indica en pantalla.

Enlace público: https://flightpredict-agent.streamlit.app/ (si la app está dormida por inactividad, pulsar «Yes, get this app back up!»; tarda ≈ 1 min).

Pestañas: (1) Demo del agente, escenario típico o de falla esperada, con tabla percepción → decisión → acción y descarga del log JSONL con marca de tiempo; (2) Vuelo manual, para controlar las entradas de un vuelo; (3) Pruebas M1–M4, que ejecuta las 65 corridas con los umbrales de config/thresholds.json y ofrece los logs y CSV en un .zip, junto a los resultados guardados del documento para comparar; (4) Arquitectura y UML, con los diagramas de la sección 9.

Casos de prueba sugeridos en la plataforma

#	Pestaña	Qué hacer	Qué debe ocurrir
1	Demo	Escenario Típico, 2 días, Ejecutar	Tabla percepción → decisión → acción de cada vuelo y su resultado real.
2	Demo	Escenario Falla esperada, 2 días	Entradas inválidas o ventana < 3 h → REVISION_MANUAL; sensor sin dato → predicción con clima imputado; 0 excepciones.
3	Vuelo manual	Valores por defecto (JFK → ORD, clima malo)	ALERTAR con probabilidad alta.

#	Pestaña	Qué hacer	Qué debe ocurrir
4	Vuelo manual	Origen ZZZ, o antelación de 1 h, o fecha no válida	REVISION_MANUAL (entrada inválida o ventana insuficiente).
5	Vuelo manual	Desmarcar «Pronóstico disponible»	El agente imputa el clima con la climatología (clima imputado = True).
6	Vuelo manual	Marcar «tramo anterior retrasado» con 60 min	Sube P(retraso): retraso en cadena (memoria de corto plazo).
7	Pruebas M1–M4	Ejecutar las 65 corridas (varios minutos)	Tabla frente a los umbrales; M3 no cumple (análisis en 9.7).

Alcance: la plataforma demuestra el ciclo del agente y permite reproducir las pruebas; sigue operando sobre el entorno simulado, por lo que no sustituye la validación con datos reales (sección 9.8).

9.10 Enlaces

Recurso	Enlace
Repositorio (GitHub)	https://github.com/HasmetG/flightpredict-agent (incluye README.md y requirements.txt)
Plataforma desplegada (Streamlit Community Cloud)	https://flightpredict-agent.streamlit.app/
Video (≤ 20 min)	No aplica (optamos por plataforma desplegada)
Logs de pruebas	results/logs/*.jsonl.gz (copias sin comprimir en results/logs_muestra/), results/vuelos_detalle.csv.gz, results/detalle_por_corrida.csv, results/tabla_resultados.csv

10. Conclusiones

FlightPredict-Agent se implementó como un agente basado en utilidad con modelo interno y aprendizaje, con un ciclo percepción → decisión → acción identificable en el código, memoria de corto y largo plazo, un registro de ejecución con marca de tiempo y un entorno simulado que respeta las propiedades de ambiente del diseño. Las pruebas, con umbrales fijados antes de ejecutar, dieron tres medidas cumplidas (exactitud de 74,0 %, antelación de 100 % y manejo de fallas de 100 %) y una incumplida (recall de 46,0 % frente a 60 %).

El hallazgo principal es que la medida de exactitud original era insuficiente: con ≈ 20 % de vuelos retrasados, la línea base «nunca alertar» ya la supera, y subir la sensibilidad del agente cuesta exactitud. El diseño debe, entonces, definir con el equipo de operaciones el costo relativo de una falsa alarma y de un retraso no anticipado, y medir el agente con una métrica que los combine. Desde la perspectiva ética se mantiene lo planteado en el diseño: el agente es un apoyo a la decisión, supervisado por humanos, y debe revisarse el sesgo por aeropuerto y aerolínea antes de usarse con datos reales.

Referencias

- Russell, S. J., & Norvig, P. Artificial Intelligence: A Modern Approach (Global Edition).
- IBM. «Components of AI agents». https://www.ibm.com/mx-es/think/topics/components-of-ai-agents
- Bureau of Transportation Statistics (BTS), U.S. Department of Transportation. Datos históricos de vuelos domésticos. https://www.transtats.bts.gov
- «Flight delay prediction: Evaluating machine learning algorithms for enhanced accuracy». PLOS ONE. https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0335141
- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- Hendrycks, D., Burns, C., Chen, A., & Ball, S. (2021). CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review. arXiv:2103.06268.

Anexo A. Extracto del registro de ejecución (S01, repetición 1, vuelo 1001-000192)

Cada línea del log (JSONL comprimido) guarda la marca de tiempo real (ts_wall), la marca de tiempo simulada (ts_sim), la fase y los datos. Se muestran la percepción, la decisión, la acción y la retroalimentación del mismo vuelo que se usa como ejemplo en 9.3 (campos largos abreviados con «…»).


```
{"ts_wall": "2026-10-17T22:47:04.925", "ts_sim": "2026-07-09T07:25:52.401996", "phase": "percepcion", "data": {"sched_dep": "2026-07-09T10:25:52.401996", "origin": "PHX", "dest": "LAS", "forecast": {"vis": 9.7179497527507, "wind": 7.641122246979663, "precip": 0.0, "temp": 24.972822728905484}}}
{"ts_wall": "2026-10-17T22:47:04.934", "ts_sim": "2026-07-09 07:25:52.401996", "phase": "decision", "data": {"plan": "[…5 pasos…]", "action": "NO_ALERTAR", "p_delay": 0.087, "d_hat": 48.354, "u_alert": -0.703, "lead_min": 180.0, "wx_imputed": false, "issues": [], "last_delay": null, "est_carry": null}}
{"ts_wall": "2026-10-17T22:47:04.934", "ts_sim": "2026-07-09 07:25:52.401996", "phase": "accion", "data": {"tipo": "SIN_ALERTA"}}
{"ts_wall": "2026-10-17T22:47:04.949", "ts_sim": "2026-07-09 10:55:41.880932", "phase": "retroalimentacion", "data": {"retraso_real_min": 29.825, "acierto": false}}
```